

---

# Ethos V1: Ethereum Forensic Tracing Tool

CMP320: Advanced Ethical Hacking

BSc Ethical Hacking Year 3

2025/26

---

*Note that Information contained in this document is for educational purposes.*

## Abstract

---

The rapid growth of the decentralised finance space has been accompanied by a significant increase in sophisticated money-laundering methods and services that exploit blockchain anonymity. These operations often begin with high-impact cybercrimes, such as the attack on a project on the Ethereum blockchain in 2023 that resulted in a six-figure loss of the project's marketing, team & development funds. Following these kinds of thefts, the challenge is the layering and obfuscation process known as a peel chain, in which the stolen capital is fragmented across a complex network of cryptocurrency wallets to mask the trail. Manual tracing of these networks in standard blockchain explorer applications is inefficient and prone to human error, underlining the clear need for automated forensics solutions.

The main aim of this project is the development of Ethos V1, a Python-based command-line tool that is designed to automate multi-hop transaction tracing. By integrating Etherscan's API, the tool utilises a breadth-first search algorithm to systematically map the outgoing transaction flows from a suspect's address. To ensure reliability and prevent data loss, the tool enforces API rate limits throughout. If the search scope is too large, the limitations of the free API would cause the application to encounter errors. This approach allows investigators to gather large-scale relational data on a suspect's address much faster than traditional manual investigation methods.

When the tool was used against the 2023 project incident data, Ethos V1 successfully identified an extensive list of related wallets & transactions linked to the layering process used to launder the stolen cryptocurrency. The tool then generates multiple files: 2 CSV ledgers and a GEXF graph file. These allow the user to have a clear visual representation of the flow of funds within 3rd-party software such as Gephi. While this version serves as a functional proof of concept for evidence gathering within blockchain forensics, it demonstrates how automation can and will significantly reduce investigative overhead. This tool provides a foundation for a future dissertation project focused on multichain tracing, with advanced heuristics for suspicious activity from scanned suspect cryptocurrency wallet addresses.

# Contents

---

|       |                                                     |
|-------|-----------------------------------------------------|
| 1     | Introduction                                        |
| 1.1   | Background                                          |
| 2     | Aims                                                |
| 3     | Methodology                                         |
| 3.1   | Research Framework and Forensic Protocol            |
| 3.2   | Technical Environment and Tool Selection            |
| 3.3   | Algorithmic Design and Justification                |
| 3.4   | Ethical Considerations                              |
| 3.5   | Use of Generative AI                                |
| 4     | Program and Development                             |
| 4.1   | Architecture and CLI Implementation                 |
| 4.1.1 | Structural Modularity & Security                    |
| 4.1.2 | Command-Line Interface Design                       |
| 4.2   | Etherscan API Integration and Rate Limit Compliance |
| 4.3   | Forensic Processing Pipeline                        |
| 4.3.1 | Transaction Graph Construction                      |
| 4.3.2 | BFS Traversal Engine                                |
| 4.3.3 | Contract and Address Labelling                      |
| 4.4   | Output Generation                                   |
| 4.4.1 | Terminal Output and Investigation Summary           |
| 4.4.2 | Node Ledger (CSV)                                   |
| 4.4.3 | Transaction Ledger (CSV)                            |
| 4.4.4 | Graph Export (GEXF)                                 |
| 4.5   | Testing and Validation                              |
| 4.5.1 | Primary Investigation Run                           |
| 4.5.2 | Validation against Etherscan                        |
| 4.5.3 | Edges – High Volume Node                            |
| 4.5.4 | Edges – Rate Limit Compliance                       |
| 4.5.5 | Edges – No Transaction Address                      |
| 4.6   | Sample Investigation Results                        |
| 5     | Discussion                                          |
| 5.1   | Overall Discussion                                  |
| 5.2   | Future Work                                         |
|       | References 28                                       |
|       | Appendices 29                                       |

# Introduction

## Background

The decentralised finance landscape is currently a fast-paced arms race between security researchers and cybercriminals. Although the blockchain is an immutable, publicly available ledger, the network's anonymity is frequently exploited to facilitate large-scale money laundering. The business problem this project addresses is the significant gap between the limitations of manual investigation methods & the speed of illicit fund dispersal.

A catalyst for this investigation was a suspected phishing attack on an Ethereum-based altcoin in 2023, which resulted in a 6-figure loss, approximately £120k, from the project's development and marketing funds. This specific incident highlights the critical issue within on-chain forensics – the layering process. The attackers will most commonly use a peel chain typology, a method in which stolen assets are transferred through an extensive network of cryptocurrency wallets. In each “hop”, the money is transferred to a further wallet (the next “hop”), service, or centralised exchange.

The complexity of this process is amplified by attackers' behaviours. In the 2023 case, the funds were moved slowly over a 6–7-hour window from the first exploit. This was then further accompanied by taunts from the attackers via Telegram, a social media platform used within the cryptocurrency community. The level of confidence stems from their awareness of the limitations of tracing and justice in addressing cryptocurrency-related cybercrime. There are applications on the market available for automated tracing, but they are mostly limited to commercial use, or are a steep learning curve, which then results in manual tracing – this is commonly done through standard blockchain explorers, which involves manually reviewing & recording each transaction, which is highly prone to human error due to the extremity of the money trail.

Ethos V1 is designed to assist in unravelling this obfuscation. By replacing hundreds of manual clicks with an automated, thorough search, the tool allows mapping the entire dispersal network in minutes – providing all addresses and transactions related to the suspect address. The birds-eye view that Ethos provides the user is essential for investigating a peel chain money laundering operation.

# Aims

The objective of this project is to architect and implement a functional forensic utility – Ethos V1 – that automates the discovery of transaction links and related wallets, reducing the key manual steps of blockchain investigation. This project seeks to assist with the gap between raw blockchain data and forensic intelligence through the following aims:

Create a robust data ingestion pipeline through the integration of Etherscan’s free tier API.

Develop a tracing algorithm based on breadth-first search that maps fund dispersal across a default 3 “hop” threshold from a provided suspect address.

Implement rate limiting and error handling to ensure the tools' stability & data integrity, given the constraints of a free-tier API.

Provide a visual forensic reconstruction of the layers of a peel chain through the generation of 2 CSV files containing information found in a scan, and a GEXF graph file that allows for third-party visualisation apps such as Gephi, which will display a thorough map of all data found.

Integrate smart contract/service endpoints via Etherscan’s free API, enabling basic labelling of services within the transaction graph.

Ensure all data collection is conducted exclusively through publicly available blockchain data in compliance with OSINT principles and without any personal data/live systems.

# Methodology

The development of Ethos V1 followed an Open-Source Intelligence (OSINT) research framework, which systematically extracts and analyses publicly available data from the Ethereum blockchain without exploiting or interacting with live systems.

## Research Framework and Forensic Protocol

The project utilises a deterministic tracing method to map the flow of illicit cryptocurrency transactions. To ensure the integrity of any findings, the tool adheres to the principles of digital forensics by maintaining an immutable record of all transactions via their cryptographic transaction hashes. This allows for any secondary investigator to verify the tool's output directly against the blockchain.

## Technical Environment and Tool Selection

Python was selected as the programming language for Ethos V1 because it is an industry-standard for developing cybersecurity tools, thanks to its extensive library ecosystem, simplicity, and versatility.

To ensure the reproducibility of the project, the specific software and libraries used are detailed below:

- Python 3.11: The core language used for the development and execution of the tool.
- Requests (v2.33.1): used to handle all communications with the Etherscan API, allowing the tool to fetch raw transaction data via HTTP GET requests.
- NetworkX(v3.6.1): used to construct and manage the Directed Graph in memory – this allows for ethos to map the links between different wallet addresses mathematically.
- Argparse: a built-in module for creating a command-line interface (CLI). This ensures that the tool functions as a terminal-based utility that requires minimal user input.
- Operating system and IDE: the project was developed on Windows 11, using Visual Studio Code as the main editor for scripting and debugging

## Algorithmic Design and Justification

The core computational engine of Ethos V1 is a breadth-first search (BFS) algorithm. This choice is a mathematically sound approach in blockchain forensics, as the blockchain is essentially an ever-expanding acyclic graph.

While a depth-first search (DFS) would force the script to follow a single chain of cryptocurrency transactions, potentially leading to getting stuck investigating and scanning large volume wallets such as a Binance exchange hot wallet, the use of BFS maps the network in “hops”. This ensures the tool captures the full radius of illicit cryptocurrency dispersal, a critical feature for identifying the branches of a peel chain.

## Ethical Considerations

The development of Ethos V1 was conducted within a strict ethical boundary. The tool operates explicitly as an OSINT instrument – all data the tool uses is publicly available on the Ethereum blockchain and accessed via Etherscan's legitimate and documented api. At no point does the tool interact with any live systems, execute transactions or exploit any vulnerability. No personal data is collected or processed, Ethereum wallet addresses are pseudonymous identifiers not personally identifiable information, and the tool makes no attempt to link these addresses to real-world identities.

The real incident used as the investigation case study involves publicly viewable on chain data – the wallet addresses involved are shown in a partially redacted form within this report as good practice. The tool is framed throughout as an educational tool and a forensic search instrument, consistent with the principles of this module. Any further development of the tool will be consistent with these too, ensuring that it is an OSINT only tool.

## Use of Generative AI

In accordance with the project guidelines regarding the use of generative AI, it is disclosed that AI tools were used as technical assistants and mentors throughout the development of Ethos V1. From the beginning of the project, AI was utilised to build the necessary programming skills and knowledge required to complete the scripting.

Specific areas where AI assistance was present:

Library implementation: used to assist with the technological logic for the network graph objects and the argparse CLI integration.

Coding understanding: helped in explaining complex sections of code to ensure a thorough understanding of the tool's functionality.

Summarisation: assisted in summarisation of the technical documentation and the organisation of the project plan.

Reporting: used for spelling and grammar checking, and for writing structure assistance.

While AI served as a supportive tool throughout the development of Ethos V1, the core conceptual idea, aims, forensic logic, and overall project objectives and outcome are all original work stemming from a strong passion for cryptocurrency.

# Program and Development

The development of Ethos V1 was completed in a deliberate and organised sequence, allowing for each phase to build on the last. The following section of the report shows this process in the order it was executed: first, establishing the architecture and UI; then implementing the data ingestion pipeline; then implementing the core forensic processing logic; and finally, generating the output. Testing and validation are documented, followed by a demonstration of the tool against the real-life 2023 incident previously mentioned.

## Architecture and CLI Implementation

Ethos V1 prioritised a modular, maintainable architecture that separates concerns, ensuring the tool is easy to build upon and audit. Python 3.11 was selected as the programming language, consistent with its established position in cybersecurity as the industry standard for tooling and its extensive library ecosystem – specifically the requests, networkX & argparse libraries, which are key assets to the tool's functionality. The architecture was kept lightweight, focusing complexity on the forensic logic.

### Structural Modularity & Security

The script was structured into two primary files to ensure that operational logic is separated from sensitive configuration:

`ethos.py`: contains all core logic, traversal algorithms, output generation and the CLI entry points.

`config.py`: stores the Etherscan API key, isolating it from the main script.

The decision to isolate the API key in a separate configuration file is a standard security practice – storing credentials such as a key directly in the main script would expose them if the code were shared, submitted, or published in a repository. By separating these, it ensures that the tool can be distributed freely without compromising the associated API account. In a professional context, the `config.py` file would be excluded from version control entirely through a `.gitignore` entry.

To maintain the tool's reproducibility for users, a `requirements.txt` file was included, allowing all necessary dependencies to be installed with a single command.

### Command-Line Interface Design

Ethos V1 was designed to be a non-GUI, terminal-based application, consistent with the operational context of forensics tools where command-line interfaces are standard. A robust CLI was created using Python's built-in `argparse` library, enabling a streamlined investigation workflow in which all required user input parameters are defined at program execution, with no interactive prompts.

Three custom arguments are used within the program:

`--address (-a)`: this is a required field – it accepts the target Ethereum cryptocurrency address as a string. This is the origin point of the investigation and the root node of the transaction graph.

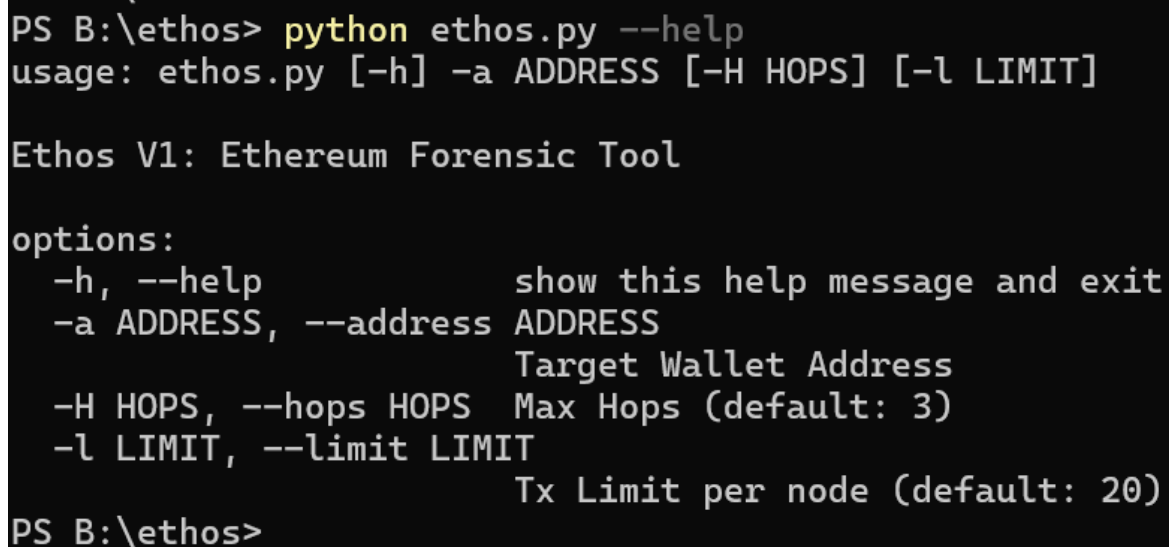


--hops (-H) this is an optional field – it defines the maximum hop depth for the BFS traversal and controls the search radius of the investigation. This defaults to three.

--limit (-l) this is also an optional field – it caps the number of transactions retrieved per node, acting as a safeguard against high volume wallets flooding the graph with unrelated data. This defaults to twenty.

Argparse automatically generates a --help flag from these definitions, providing users with a formatted description of available arguments in the terminal without having to inspect the code.

Upon execution, the tool automatically displays a branded ASCII art banner featuring Ethos V1's name, version, and catchphrase alongside the Ethereum logo. This helps to establish a clear visual identity and confirms to the user they are using the correct tool & version before any investigation takes place.



```
PS B:\ethos> python ethos.py --help
usage: ethos.py [-h] -a ADDRESS [-H HOPS] [-l LIMIT]

Ethos V1: Ethereum Forensic Tool

options:
  -h, --help            show this help message and exit
  -a ADDRESS, --address ADDRESS
                        Target Wallet Address
  -H HOPS, --hops HOPS  Max Hops (default: 3)
  -l LIMIT, --limit LIMIT
                        Tx Limit per node (default: 20)
PS B:\ethos>
```

[Figure 4-1: Terminal screenshot showing the --help output with all available CLI arguments and descriptions]

## Etherscan API Integration and Rate Limit Compliance

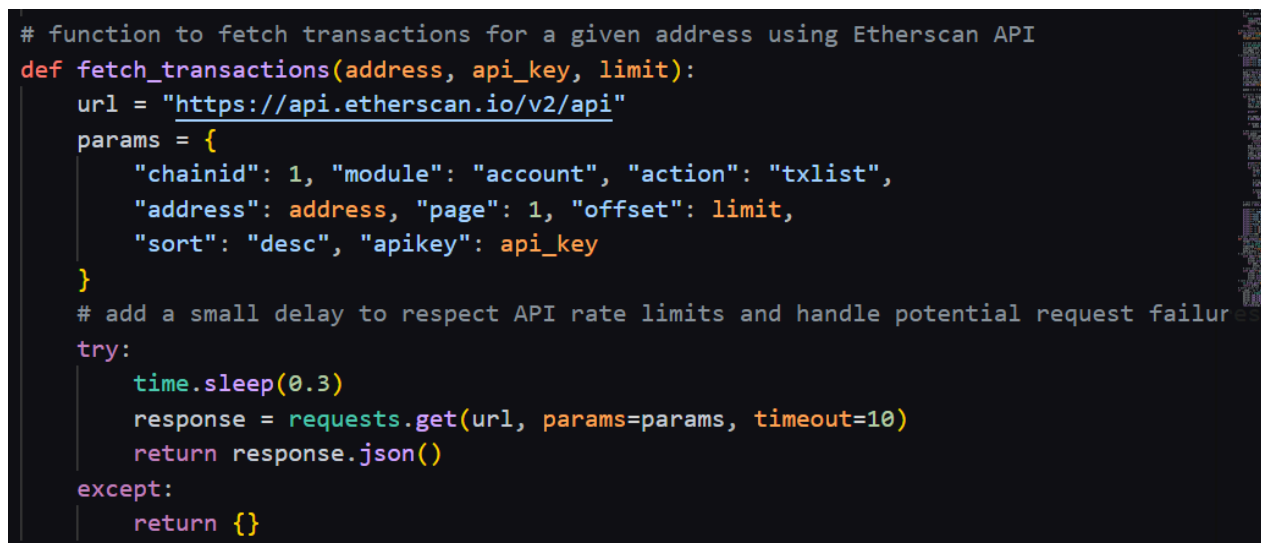
Ethos V1 was engineered to replace manual, error-prone searches of blockchain explorers with an automated data ingestion pipeline. To achieve this, the tool integrates Etherscan's API to harvest raw transaction details directly from the Ethereum ledger at scale.

This API implementation utilises the requests library to execute HTTP GET requests against the Etherscan API. Specifically, the txlist endpoint retrieves outgoing transaction history for the user-supplied wallet address. This API call explicitly uses chain ID 1, ensuring all queries are within the scope of the Ethereum mainnet and that no testnet transaction data is retrieved.

The main technical challenge in integrating the Etherscan API was the 5 calls-per-second limit imposed by the free usage plan. To ensure the tool remained compliant and did not trigger rate limiting, a `time.sleep(0.3s)` was added to every API request using Python's built-in time module. This created a 300 millisecond pause between calls, ensuring the tool never exceeds the API's restraints regardless of the

scope. Additionally, the tool automatically converts all transaction values retrieved from Wei – etherscan's native denomination, which represents the smallest unit of Ether – into a readable ETH value by dividing by  $10^{18}$ , ensuring the output is immediately interpretable without manual conversion.

Beyond rate limit compliance, a key consideration was ensuring that a single failed API call would not prematurely terminate an investigation. Both the transaction-fetching and contract-labelling functions wrap API calls in try/except blocks – if a request times out or returns an incorrect response, the tool will not record any data for that address and will continue traversal. A ten-second request timeout is also set on all calls to stop the tool from continuing on a slow or unresponsive server. This approach ensures continuity of investigation and data integrity. In a forensic context, losing all gathered data due to a single network or API error would be an unacceptable outcome.



```
# function to fetch transactions for a given address using Etherscan API
def fetch_transactions(address, api_key, limit):
    url = "https://api.etherscan.io/v2/api"
    params = {
        "chainid": 1, "module": "account", "action": "txlist",
        "address": address, "page": 1, "offset": limit,
        "sort": "desc", "apikey": api_key
    }
    # add a small delay to respect API rate limits and handle potential request failure
    try:
        time.sleep(0.3)
        response = requests.get(url, params=params, timeout=10)
        return response.json()
    except:
        return {}
```

[Figure 4-2: Screenshot of the `fetch_transactions` function demonstrating the API call, `time.sleep` delay, and try/except error handling]

## Forensic Processing Pipeline

Once raw transaction data has been retrieved from the Etherscan API, it then enters the forensic processing pipeline. The transaction graph is initialised first, then during the traversal, each newly discovered address is added to the graph, evaluated for contract labelling, and queued for further exploration. These three components are interlinked by design, ensuring that by the time the traversal completes, the graph is fully constructed, annotated, and prepared for export.

## Transaction Graph Construction

The in-memory representation of the transaction network is built through NetworkX's directed graph (DiGraph) object. Directed graphs were chosen over undirected graphs because cryptocurrency transactions have an explicit direction – value moves from a source address to a destination address - and this relationship is forensically significant. Reversing this direction would remove critical investigative context about how funds were moved through the network.

On initialisation, the graph object (G) is created, and the target address is registered as the root node at hop depth zero. As the traversal continues, each newly discovered wallet address is added as a node with three metadata attributes: the hop depth from the origin, the resolved contract label, and a fresh classification tag. Each transaction between two addresses is recorded as a directed edge, holding the transferred value in ETH and the cryptographic transaction hash to verify each edge against the blockchain.

A “fresh” attribute is assigned to each node at the point of discovery. The investigations target address is assigned to fresh=“False”, as it is the known starting point of the process. All subsequent addresses are assigned fresh=“True”, showing they were identified as new connections within the scan. This is useful to the user reviewing the node ledger to immediately distinguish between the suspect wallet and newly found wallet addresses during the investigation.

This graph structure serves as the central data object throughout the tool; all traversal logic, CSV exports, and the final GEXF output file are derived directly from it, ensuring a consistent data source across all outputs.

```
def run_trace(target_address, max_hops, tx_limit):
    api_key = config.ETHERSCAN_API_KEY
    target_address = target_address.lower()

    # graph and data structures to hold nodes, edges, and visited status
    G = nx.DiGraph()
    visited_nodes = {}
    all_edges = []
    total_eth_out = 0
```

[Figure 4-3: Screenshot of the graph construction code showing DiGraph initialisation]

## BFS Traversal Engine

The traversal engine of Ethos V1 is implemented as a breadth-first search across the transaction graph. As justified within the methodology, BFS was chosen to ensure the tool maps the full dispersal of funds at each hop level before going deeper. This is the correct approach when investigating peel chains, as the branching structure of a laundering operation is more forensically valuable than any single deep chain.

The target address provided by the user is intentionally processed outside the main traversal loop – its outgoing transactions are retrieved and recorded first, then the discovered destination addresses are loaded into the queue at the first hop depth. This ensures the origin node is treated as the starting point of the investigation rather than a discovered address, preserving forensic distinction between the suspect wallet address and all other wallets identified.

The algorithm is then managed via a standard FIFO queue. The main loop processes each queued address in turn- fetching its transactions, recording all outgoing edges, resolving any contract label available, then putting any newly discovered addresses into the queue at the incremented hop depth. The visited\_nodes dictionary serves as a protective measure for traversal – before the tool processes any address, it checks

whether it has already been visited and skips it if so. This is a critical design feature, given that the transaction graph may not be acyclic. Without this guard, the tool could enter an infinite loop or produce duplicate edges, corrupting the forensic output.

The maximum hop depth argument, set to 3 by default, serves as the outer boundary of the investigation. This is a deliberate design choice, as the majority of peel chain activity occurs within the first three hops of the origin address, and extending the search beyond three hops risks mass node discovery, which could exceed the free API usage limits. The transaction limit per node provides a further safeguard, limiting the number of transactions retrieved from any single address. This prevents a high-volume wallet, such as a Centralised exchange hot wallet, from flooding the graph with irrelevant data.

```
# BFS traversal for subsequent hops
while queue:
    current_address, current_hop = queue.pop(0)
    if current_address in visited_nodes or current_hop > max_hops:
        continue
    # fetch transactions for the current address
    data = fetch_transactions(current_address, api_key, tx_limit)
    transactions = data.get('result', [])
    # get contract label for current address and add to graph
    label = get_contract_label(current_address, api_key)
    visited_nodes[current_address] = {'hop': current_hop, 'label': label, 'fresh': "True"}
    G.add_node(current_address, hop=current_hop, label=label, fresh="True")

    print(f"\n[*] Processing Hop {current_hop}: {current_address[:14]}...")
    # process transactions for current address and queue next addresses
    for tx in transactions:
        target = tx['to'].lower()
        if not target: continue
        val = int(tx['value']) / 10**18

        # only outgoing transactions from current address
        all_edges.append({'from': current_address, 'to': target, 'value': val, 'hash': tx['hash'], 'hop': current_hop})
        G.add_edge(current_address, target, value=val, hash=tx['hash'])

        # queue next address if not visited and within hop limit
        if target not in visited_nodes:
            print(f"    [Link] {current_address[:8]}... -> {target[:8]}... | {val:.4f} ETH")
            queue.append((target, current_hop + 1))
```

[Figure 4-4: Screenshot of the BFS queue loop showing the while loop, hop depth check, visited node guard, and queue population logic]

## Contract and Address Labelling

A key analytical feature of Ethos V1 is the ability to automatically identify if a discovered address belongs to a known smart contract or named service on the Ethereum network. Unlike a post-processing step, the contract labelling is executed in-line during the BFS traversal. The moment a new address is discovered, `get_contract_label` is called immediately & any resolved label is written directly into the graph node before traversal continues. This ensures the graph is fully annotated by the time the traversal completes.

The labelling is achieved through a secondary API call to Etherscan's `getsourcecode` contract endpoint, which returns the contract name if the address's source code is verified on the explorer. In a forensic context, this is a meaningful feature. When tracing a peelchain, it is highly significant whether stolen funds have entered a known decentralised/centralised exchange, a decentralised protocol or a privacy-focused

mixer service. Each of these implies different stages of a laundering process, and carries different implications for an investigation. By automatically labelling these nodes when discovered, the user has immediate context without having to manually look up each address.

This function is intentionally lightweight; it executes a single api call per address, respects rate limits, and falls back to an “unknown” label for any address that returns no relevant data. This will be the case for the majority of discovered addresses during an investigation, since most nodes are standard wallet addresses. Having a fallback ensures the tool doesn’t crash or corrupt data when no label is found.

```
def get_contract_label(address, api_key):
    url = "https://api.etherscan.io/v2/api"
    params = {"chainid": 1, "module": "contract", "action": "getsourcecode", "address": address, "apikey": api_key} # etherscan
    try:
        time.sleep(0.2)
        res = requests.get(url, params=params).json()
        if res.get("status") == "1" and res["result"][0].get("ContractName"): #if api call is successful and contract name is
            return res["result"][0]["ContractName"]
    except:
        pass
    return "Unknown" # if no label is found or API call fails, return "Unknown" as default label for the node in the graph
```

[Figure 4-5: Screenshot of the `get_contract_label` function showing the API call, contract name extraction, and Unknown fallback]

## Output Generation

On completion of the traversal, Ethos V1 generates 4 different outputs: a terminal summary, two structured CSV ledgers, and a GEXF graph file. Each output serves a different purpose within an investigation and is designed to be useful independently or in combination with the others.

### Terminal Output and Investigation Summary

Ethos V1 produces real-time terminal output throughout an investigation, serving as an immediate, human-readable record of the traversal's current progress. Upon execution, the tool displays a branded ASCII banner art featuring the Ethos V1 name, version, and catchphrase, alongside an Ethereum logo. This establishes a clear visual identity and confirms to a user that they are using the correct tool and version before they proceed with an investigation.

As each transaction link is discovered throughout the traversal, the tool prints the source address, destination address and the transferred ETH value to the terminal. This allows users to monitor the scope of the peel chain as it unfolds in real time, rather than waiting for the scan to complete to see results.

After completing a traversal, the tool outputs a formatted investigation summary to the terminal, reporting the total number of unique addresses discovered, the total number of transactions mapped & the total ETH flow recorded from the target address. This printed summary provides an immediate, detailed review of the scale of the laundering operation before the user opens any further exported files.

PS B:\ethos> python ethos.py -a 0xDdA24103e40781B021bbb0da47270a024224e593



Ethos V1 - Illuminating the Ledger



```
[!] INVESTIGATION START: 2026-05-02 07:57:32
[!] Target Address: 0xdda24103e40781b021bbb0da47270a024224e593
[!] Config: 3 Hops | 20 Tx Limit per node
```

```
[*] Initial Scan: Found 20 transactions leaving target.
```

```
[Link] Source -> 0xfaf2f8ded9... | Amount: 0.0128 ETH
[Link] Source -> 0xad8fca418e... | Amount: 5.2000 ETH
[Link] Source -> 0x8dc0b7b50d... | Amount: 5.5400 ETH
[Link] Source -> 0x43a7e39279... | Amount: 5.0000 ETH
[Link] Source -> 0xf06587b7ff... | Amount: 5.8639 ETH
[Link] Source -> 0x06fecb1599... | Amount: 5.0000 ETH
[Link] Source -> 0x2ce01beed... | Amount: 5.0000 ETH
[Link] Source -> 0xa4724e2b09... | Amount: 5.0000 ETH
[Link] Source -> 0x48551b6e1f... | Amount: 0.0823 ETH
[Link] Source -> 0xdfddeddb5f... | Amount: 5.0000 ETH
[Link] Source -> 0xc4176426f2... | Amount: 5.0000 ETH
[Link] Source -> 0x85316fe3e9... | Amount: 5.0000 ETH
[Link] Source -> 0x8fab6c61b56... | Amount: 5.0000 ETH
[Link] Source -> 0x5540b26f44... | Amount: 5.0000 ETH
[Link] Source -> 0xce912cdb25... | Amount: 5.0000 ETH
[Link] Source -> 0x2b9988f16b... | Amount: 5.0000 ETH
[Link] Source -> 0x3fc91a3afd... | Amount: 0.0000 ETH
[Link] Source -> 0x45804880de... | Amount: 0.0000 ETH
[Link] Source -> 0x3fc91a3afd... | Amount: 0.0000 ETH
[Link] Source -> 0x2402253996... | Amount: 0.0000 ETH
```

```
[*] Processing Hop 1: 0xfaf2f8ded954...
[Link] 0xfaf2f8... -> 0x077d36... | 0.0116 ETH
```

```
[*] Processing Hop 1: 0xad8fca418eb8...
[Link] 0xad8fca... -> 0x18c4bf... | 5.1990 ETH
```

```
[*] Processing Hop 1: 0x8dc0b7b50db2...
[Link] 0x8dc0b7... -> 0x754ab5... | 5.5395 ETH
```

```
[*] Processing Hop 1: 0x43a7e39279d1...
[Link] 0x43a7e3... -> 0x18c4bf... | 4.9993 ETH
```

```

[Link] 0xf06587... -> 0xa9d1e0... | 1.0000 ETH
[*] Processing Hop 1: 0xf06587b7ff1e...
[Link] 0xf06587... -> 0xa9d1e0... | 1.0000 ETH
[Link] 0xf06587... -> 0xb5d85c... | 0.3685 ETH
[Link] 0xf06587... -> 0xa9d1e0... | 0.4856 ETH
[Link] 0xf06587... -> 0xb5d85c... | 0.5000 ETH
[Link] 0xf06587... -> 0xa9d1e0... | 0.5000 ETH
[Link] 0xf06587... -> 0xb5d85c... | 0.2000 ETH
[Link] 0xf06587... -> 0xa9d1e0... | 0.5000 ETH

[*] Processing Hop 1: 0x06fecb159974...
[Link] 0x06fecb... -> 0xee7f2d... | 4.9995 ETH

[*] Processing Hop 1: 0x2ce01beeed98...
[Link] 0x2ce01b... -> 0xee7f2d... | 4.9996 ETH

[*] Processing Hop 3: 0x0eafe9084169...
[Link] 0x0eafe9... -> 0x9d9827... | 0.0360 ETH
[Link] 0x0eafe9... -> 0x9d9827... | 0.2700 ETH
[Link] 0x0eafe9... -> 0x9d9827... | 0.1934 ETH
[Link] 0x0eafe9... -> 0x7d05fa... | 0.0030 ETH
[Link] 0x0eafe9... -> 0x0ea1a2... | 0.0010 ETH

[*] Processing Hop 3: 0x4bbcce510d1e...
[Link] 0x4bbcce... -> 0x72bee3... | 0.0016 ETH
[Link] 0x4bbcce... -> 0xce3a52... | 0.0066 ETH

=====
--- FINAL INVESTIGATION SUMMARY ---
Total Unique Addresses Found: 88
Total Transactions Mapped: 1308
Total ETH Flow from Target: 71.6989 ETH
-----

Files Created in 'forensic_results/':
-> 0xdda24103_20260502_0803_nodes.csv
-> 0xdda24103_20260502_0803_edges.csv
-> 0xdda24103_20260502_0803_graph.gexf
=====
PS B:\ethos> |

```

[Figure 4-6: Terminal output showing the Ethos banner, real-time link discovery during traversal, and the final formatted investigation summary block]

## Node Ledger (CSV)

The first exported file is the node ledger, written as a CSV to the /forensics\_results/ directory. All files are prefixed with the first 10 characters of the target address and a timestamp at the front in the format YYYYMMDD\_HHMM – this naming scheme ensures that outputs from different investigations are distinguishable and that subsequent scans of the same address do not overwrite.

The node ledger provides a complete list of all unique wallet addresses discovered during an investigation – each row contains 4 fields: the wallet address, the hop depth from origin, any resolved contract labels & the fresh classification flag. This file serves as the primary address ledger for an investigation – it allows anyone with access to the results to quickly identify which addresses are newly surfaced wallets, whether they are using any known services, and at what depth from the origin each address sits within the mapped network.



|    | A          | B   | C          | D     | E |  |
|----|------------|-----|------------|-------|---|--|
| 1  | address    | hop | label      | fresh |   |  |
| 2  | 0xdda24103 | 0   | Unknown    | FALSE |   |  |
| 3  | 0xfaf2f8de | 1   | Unknown    | TRUE  |   |  |
| 4  | 0xad8fca4  | 1   | Unknown    | TRUE  |   |  |
| 5  | 0x8dc0b7b  | 1   | Unknown    | TRUE  |   |  |
| 6  | 0x43a7e39  | 1   | Unknown    | TRUE  |   |  |
| 7  | 0xf06587b  | 1   | Unknown    | TRUE  |   |  |
| 8  | 0x06fecb1  | 1   | Unknown    | TRUE  |   |  |
| 9  | 0x2ce01be  | 1   | Unknown    | TRUE  |   |  |
| 10 | 0xa4724e2  | 1   | Unknown    | TRUE  |   |  |
| 11 | 0x48551b6  | 1   | Unknown    | TRUE  |   |  |
| 12 | 0xdfddedd  | 1   | Unknown    | TRUE  |   |  |
| 13 | 0xc417642  | 1   | Unknown    | TRUE  |   |  |
| 14 | 0x85316fe  | 1   | Unknown    | TRUE  |   |  |
| 15 | 0x8fabcb1  | 1   | Unknown    | TRUE  |   |  |
| 16 | 0x5540b26  | 1   | Unknown    | TRUE  |   |  |
| 17 | 0xce912cc  | 1   | Unknown    | TRUE  |   |  |
| 18 | 0x2b9988f  | 1   | Unknown    | TRUE  |   |  |
| 19 | 0x3fc91a3  | 1   | UniversalF | TRUE  |   |  |
| 20 | 0x4580488  | 1   | AdminUpg   | TRUE  |   |  |
| 21 | 0x2402253  | 1   | PaxGold    | TRUE  |   |  |
| 22 | 0x077d360  | 2   | Unknown    | TRUE  |   |  |

<
>
0xdda24103\_20260502\_0803\_nodes

[Figure 4-7: Sample rows from the generated \_nodes.csv output showing address, hop, label, and fresh fields]

## Transaction Ledger (CSV)

The second exported file is the transaction ledger, which records all individual transaction links mapped by the tool during an investigation. Each row has 5 fields: source address, destination address, transfer value in ETH, the cryptographic transaction hash and the hop depth at which the link was discovered.

Including the transaction hash is also a deliberate design choice. Each hash is a fully unique identifier that points directly to the original entry on the Ethereum blockchain – it provides a verifiable reference for every data point the tool produces, allowing validation of the tool's output against the public ledger without rerunning the scan.

| from       | to         | value   | hash                                    | hop |
|------------|------------|---------|-----------------------------------------|-----|
| 0xdda24103 | 0xfaf2f8de | 0.01276 | 0x38ac6f86435db9e53b67db0df9b7586b0e2   | 0   |
| 0xdda24103 | 0xad8fca4  | 5.2     | 0x6f50392f86dc15fcb1e57c280161fe6aab3e  | 0   |
| 0xdda24103 | 0x8dc0b7f  | 5.54    | 0x485e13e2654790f8f0459a1d381955138ck   | 0   |
| 0xdda24103 | 0x43a7e39  | 5       | 0x3c0bc78d589117469ca5548403b62c6cb1    | 0   |
| 0xdda24103 | 0xf06587b  | 5.8639  | 0x54a8666a077c6a846d002e84105b2f4efa0   | 0   |
| 0xdda24103 | 0x06fecb1  | 5       | 0x80101373c19849f2e595bf75932fc0ac6ad   | 0   |
| 0xdda24103 | 0x2ce01be  | 5       | 0x673eef68a6c7736c9acb88ac959dcd20620   | 0   |
| 0xdda24103 | 0xa4724e2  | 5       | 0x517ec639ccd33d699b463197b51925a057    | 0   |
| 0xdda24103 | 0x48551b0  | 0.08229 | 0x4fe45d7fbbd98bb0686172dc79c719cf018   | 0   |
| 0xdda24103 | 0xdfddedd  | 5       | 0x4984eac1fd71d149522dc842cf8bff6f5907  | 0   |
| 0xdda24103 | 0xc417642  | 5       | 0xac4d8eb2e4d05debed8859726206b842d3    | 0   |
| 0xdda24103 | 0x85316fe  | 5       | 0x3e2a72ad4faed9935fc43fec5f43f1cf75adb | 0   |
| 0xdda24103 | 0x8fabcb61 | 5       | 0x73651aff38b5fa74d7aee6a239d6625f130c  | 0   |
| 0xdda24103 | 0x5540b20  | 5       | 0x78231b783a4195ebf83413b9243c27da81    | 0   |
| 0xdda24103 | 0xce912cc  | 5       | 0xa551b799444a41de3b9425e9110c165224    | 0   |
| 0xdda24103 | 0x2b9988f  | 5       | 0x8fa79cfb3a5363b7230e32e2d338d9009fb   | 0   |
| 0xdda24103 | 0x3fc91a3  | 0       | 0x748900c1ae7f835c14133712444f5ad83c0   | 0   |
| 0xdda24103 | 0x458048f  | 0       | 0xd0babd9c893ca68d79b9cc2c734853a09b    | 0   |
| 0xdda24103 | 0x3fc91a3  | 0       | 0xc0bbc4b6b0c7d583c93c5b5d5c1bed379b    | 0   |
| 0xdda24103 | 0x240225f  | 0       | 0x7f4f3d7c3fd33c84d8f0cc3567b345c3db3c  | 0   |
| 0xfaf2f8de | 0x077d360  | 0.01159 | 0xfe403271ec77ea887aa11cf103d44d525d3   | 1   |
| 0xfaf2f8de | 0xfaf2f8de | 0.01276 | 0x38ac6f86435db9e53b67db0df9b7586b0e2   | 1   |
| 0xad8fca4  | 0x18c4bf7  | 5.19895 | 0x3e0c865429cfc4656bd42c1c3fced617f660  | 1   |
| 0xad8fca4  | 0xad8fca4  | 5.2     | 0x6f50392f86dc15fcb1e57c280161fe6aab3e  | 1   |
| 0x8dc0b7f  | 0x754ab5f  | 5.53951 | 0x40b91128357bf5cf05bc59667e93d685c07   | 1   |
| 0x8dc0b7f  | 0x8dc0b7f  | 5.54    | 0x485e13e2654790f8f0459a1d381955138ck   | 1   |
| 0x43a7e39  | 0x10e4b5f  | 4.00000 | 0x415f003f2055e0d374e1d7505706f2e7e0    | 1   |

<
>
0xdda24103\_20260502\_0803\_edges
+

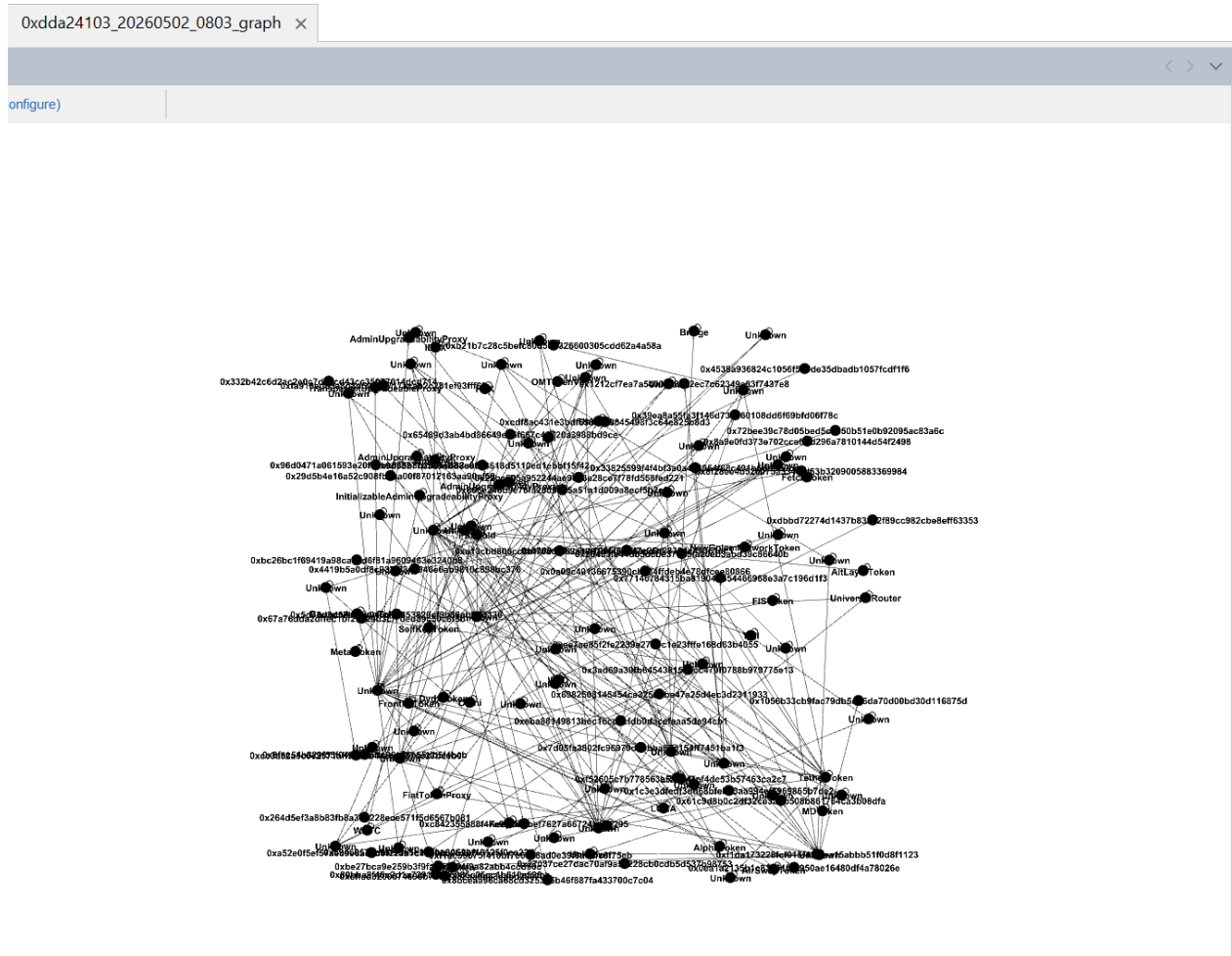
[Figure 4-8: Sample rows from the generated \_edges.csv output showing source, destination, value, hash, and hop fields]

## Graph Export (GEXF)

The most beneficial output produced by Ethos V1 is the Graph Exchange XML Format (GEXF) file. This file encodes the entire transaction graph – all nodes & edges with their associated metadata – in a structured format that can be imported into third-party graph visualisation software such as Gephi.

When rendered in Gephi, the GEXF file produces a visual map which displays the entire fund dispersal network, with the branching structure of the peel chain visible. The target address appears as the origin node, with branches spreading outwards – each representing a separate set of wallets receiving portions of the stolen capital. Node colouring, sizing, and more can be applied in Gephi using metadata attributes exported by Ethos, such as wallet addresses and contract labels, allowing users to identify which nodes represent wallets or services.

This output fulfils the aim of providing a visual forensic reconstruction of the layering process of a peel chain – a result that would be practically impossible to produce manually in a comparable timeframe using a blockchain explorer application.



[Figure 4-9: Gephi visualisation of the GEXF graph file from the 2023 investigation run, showing the full dispersal network from the origin address]

## Testing and Validation

To validate the functionality of Ethos V1, a series of tests of the key components was conducted against the tool. All tests were defined with an expected outcome before they were performed, and the actual results were compared with the expected.

## Primary Investigation Run

The expected outcome was that the tool would initialise correctly, traverse all relevant nodes as it should, and produce all four outputs without error. The tool was run against the suspected attacker address of the 2023 incident using the default configuration of 3 hops and 20 transactions per node – the actual result matched the expected outcome. The traversal completed successfully, the terminal summary reported correct data and totals and the three file outputs were generated within the /forensic\_results/ directory as expected.

```
[*] Processing Hop 3: 0x0eafe9084169...
  [Link] 0x0eafe9... -> 0x9d9827... | 0.0360 ETH
  [Link] 0x0eafe9... -> 0x9d9827... | 0.2700 ETH
  [Link] 0x0eafe9... -> 0x9d9827... | 0.1934 ETH
  [Link] 0x0eafe9... -> 0x7d05fa... | 0.0030 ETH
  [Link] 0x0eafe9... -> 0x0ea1a2... | 0.0010 ETH

[*] Processing Hop 3: 0x4bbcce510d1e...
  [Link] 0x4bbcce... -> 0x72bee3... | 0.0016 ETH
  [Link] 0x4bbcce... -> 0xce3a52... | 0.0066 ETH

=====
--- FINAL INVESTIGATION SUMMARY ---
Total Unique Addresses Found: 88
Total Transactions Mapped:    1308
Total ETH Flow from Target:  71.6989 ETH
-----
Files Created in 'forensic_results/':
-> 0xdda24103_20260502_0803_nodes.csv
-> 0xdda24103_20260502_0803_edges.csv
-> 0xdda24103_20260502_0803_graph.gexf
=====
PS B:\ethos> |
```

[Figure 4-10: Terminal summary output from the primary investigation run]

71.6989

## Validation against Etherscan

The expected outcome was that the source and destination addresses, along with the ETH value for each transaction in the exported edge CSV file, would match the data on Etherscan when the transaction hash is looked up in a blockchain explorer. A few selected transactions from the edge CSV were manually looked

up on Etherscan using the recorded transaction hash. In each test, the data matched the tool's output exactly, confirming that the API integration, data parsing, and the Wei-ETH unit conversion are all working as they should.

0xad8fca40x18c4bf7 5.198954 0x3e0c865429cfc4656bd42c1c3fced617f660b66791e870d58ccd3415a2adee

Transaction Hash:

0x3e0c865429cfc4656bd42c1c3fced617f660b66791e870d58ccd3415a2adee2f

Status:

Success

Block:

17771520

7234603 Block Confirmations

Timestamp:

1011 days ago | Jul-25-2023 05:12:23 PM +UTC

[Figure 4-11: Side-by-side comparison of a sampled hash from the edge CSV and etherscan validation ]

## Edges – High Volume Node

The expected outcome was that when a high-volume address was found, the transaction limit function would successfully cap the retrieval of tx's and prevent the graph from being flooded with irrelevant data. Throughout the tests, several addresses with higher transaction volumes were found – the transaction limit was confirmed to successfully cap the retrieval to the default 20 transactions per node, validating that the safeguard works. This is visible in Figure 4-10, where the tool processes these nodes without stalling or flooding an investigation.

## Edges – Rate Limit Compliance

The expected outcome of this test was that the tool would successfully complete a large-scale investigation without receiving any rate-limiting errors from the Etherscan API. A deliberate testing of an increased hop count to generate a larger volume of API calls – the time.Sleep delays were confirmed to prevent any rate limit errors throughout the entire run. The tool completed the investigation without any crashes or interruptions, validating the compliance logic and demonstrating that it can sustain large investigations without triggering API bans or errors.

## Edges – No Transaction Address

The expected outcome of this test was that the tool would successfully handle an address with no outgoing transactions without any errors or termination of the investigation. The tool was tested against an address with no recorded outgoing transactions. The empty result returned by the API was correctly handled; the addresses were added to the visited nodes dictionary with no edges recorded, and traversal continued on the next queued address without error or termination.

## Sample Investigation Results

To demonstrate Ethos V1's forensic capabilities, the tool was used against a wallet address associated with the 2023 Ethereum project incident described in this report. The target suspect address was used as the investigation origin, with the default configurations of 3 hops maximum and a transaction limit of 20 per

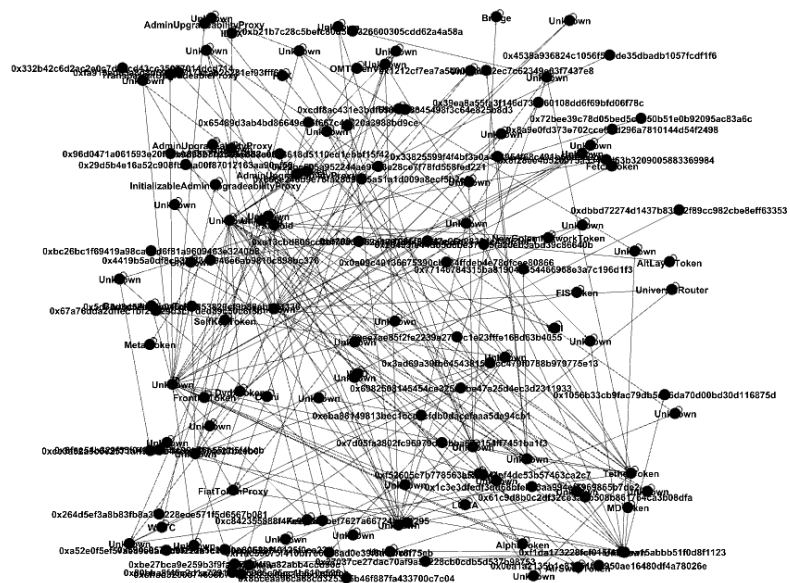
node.

The investigation gave the following:

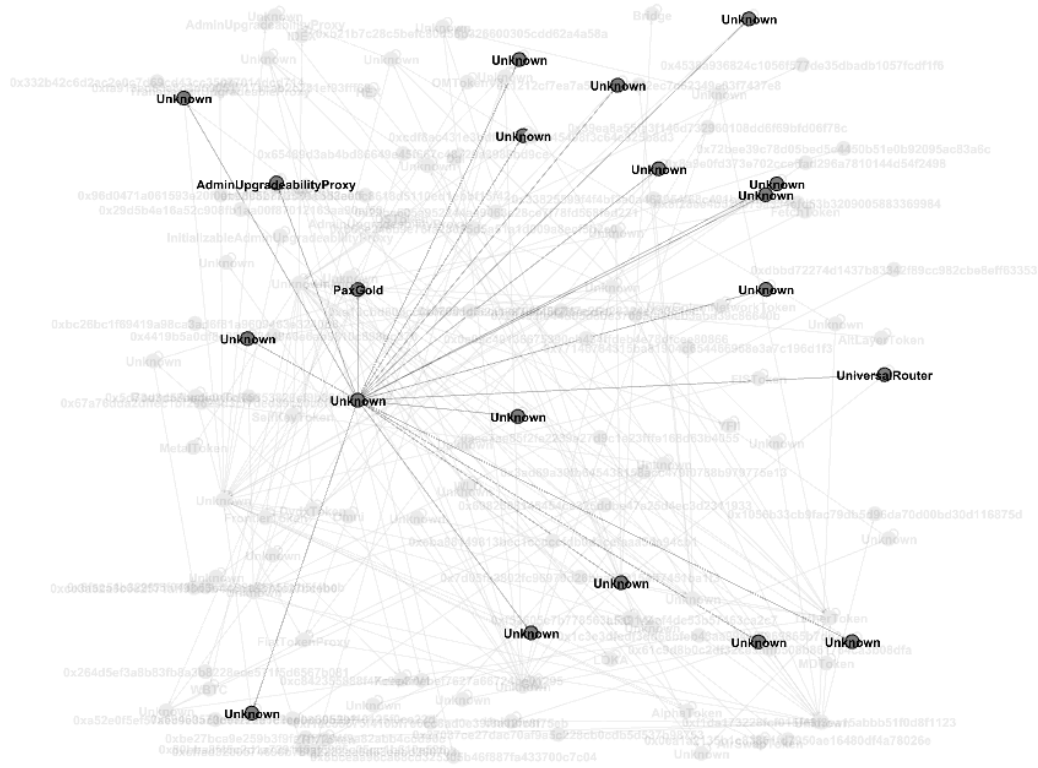
| Metric                            | Result      |
|-----------------------------------|-------------|
| Total Unique Addresses Discovered | 88          |
| Total Transactions Mapped         | 1308        |
| Total ETH Flow from Target        | 71.6989 ETH |
| Maximum Hops Configured           | 3           |
| Transaction Limit Per Node        | 20          |
| Approximate Runtime               | 5 minutes   |

The graph visualisation generated from the exported GEXF file, when rendered in Gephi, clearly illustrates the peel chain structure of this specific laundering operation. The target address is shown as the origin node, with multiple dispersal branches spreading outwards – each showing a separate chain of wallets receiving portions of the stolen capital. Several nodes within the graph were automatically labelled via the contract labelling feature, showing known services, contracts, and more encountered throughout the layering process, which provides forensic context at those specific points within the network.

The tool successfully mapped this network in [insert runtime] – this demonstrates the investigative value described in the project aims, as the same task done manually would take extensive time and effort to reproduce the same data using a standard blockchain explorer.







[Figure 4-12: Gephi visualisation of the full transaction graph from the 2023 investigation run, including zoomed in chain]



# Discussion

## Overall Discussion

Ethos V1 was developed to address a clear problem in blockchain forensics – the gap between the speed at which illicit money is dispersed across the Ethereum blockchain and the limitations of manual investigation methods. The 2023 incident that motivated this project illustrates this gap directly: a six-figure theft followed by a methodical peel chain operation that would've required hundreds of manual searches to trace via a standard blockchain explorer. The main question this project sought to answer was whether that process could be automated reliably within the constraints of a free-tier API and a compact Python codebase, and the results demonstrate that it can.

Each aim defined at the beginning of the project was met. The automated ingestion pipeline successfully replaced the manual blockchain explorer searching, the BFS traversal correctly maps out the full dispersal radius of a peel chain with the configured boundaries, rate limit compliance ensured no data loss throughout an investigation, the contract labelling feature identifies known services automatically in the graph, and all outputted files collectively create a complete, verifiable forensic record that any further investigators could work from directly.

The choice of BFS as the core traversal algorithm proved successful in practice. The peel chain structure shown from the 2023 incident is a wide dispersal with clear branching- the attacker split funds across multiple wallets at each hop rather than sending them down a single chain of transactions. A DFS algorithm would have followed a single branch to its endpoint before backtracking, potentially creating a misleading and unclear picture of a dispersal network. BFS captured the entire radius at each hop level, exactly what forensic mapping of this kind of operation needs.

There are meaningful limitations that must be acknowledged honestly; the most significant being the dependency on Etherscan's free api tier. The five calls per second limit and the transaction retrieval limits mean that Ethos V1 operates with an incomplete view of any address with a large volume of transactions – this is a trade-off of stability instead of completeness, but it means the graph produced is a sample of a dispersal network rather than a fully complete map. A paid API tier would remove these constraints and allow an uncapped retrieval.

The contract labelling feature has a similar limitation. It identifies whether addresses found by the tool have verified source code on Etherscan, as do addresses for well-known protocols, services, and exchanges. The majority of the addresses found by the tool are standard cryptocurrency wallets that will return no label, and certain money laundering operations will use unidentifiable services to evade detection. The “Unknown” label indicates that nothing was found, but it doesn't tell an investigator whether it is anything beyond an unverified contract.

Finally, it is worth reiterating the ethical boundaries within which this tool operates. Ethos V1 interacts exclusively with publicly available information via a legitimate blockchain explorer API. It does not attempt to deanonymise individuals, interact with live systems or collect any personal data. It is an

evidence-gathering tool with a mapping utility, not used for exploitation or surveillance.

## Future Work

Ethos V1 is a functional proof of concept, and its architecture was designed to allow for extensibility. Several enhancements are identified for a future, more advanced tool, most notably as a foundation of a planned dissertation-level project.

### ***Multi-chain support:***

The current tool is scoped exclusively to the Ethereum mainnet network. A large portion of modern laundering operations cross between chains, moving funds from network to network, such as Monero, Arbitrum, Polygon, Solana, or Bitcoin. They do this to increase the difficulty of tracing their money, which often makes tracing more difficult when using tools such as Ethos V1 in its current form. Extending the tool to support multi-chain traversal via other blockchain explorer APIs would significantly increase forensic coverage and is the highest-priority enhancement for a future version.

### ***Advanced heuristic classification:***

Instead of solely relying on Etherscan's contract labels, a future version could incorporate a broader address intelligence layer. This could include integration of public databases containing known mixer/exchange addresses, cross referencing OFAC sanction lists or behavioural characteristics that flag addresses with patterns consistent with tumbling or layering such as an empty wallet receiving and then immediately sending funds across a large amount of recipients in a short amount of time – this would help transform the contract labelling feature from a basic lookup to an advanced classification of wallets.

Another future classification feature could be to enhance the “fresh” flag. Currently, it simply indicates whether an address was discovered during the current scan, rather than serving as the investigation origin. This could be extended by using the Etherscan API to retrieve the first timestamp for each found address, allowing the tool to determine whether the wallet was newly created around the time of an incident. In a peel-chain investigation, a large number of newly created wallets is a significant forensic indicator that suggests the attacker used a network of pre-created wallets specifically for the laundering operation. This would transform the “fresh” flag into an important forensic investigation indicator.

### ***Mixer & privacy protocol detection:***

Privacy protocols and mixer services such as Tornado Cash pose a significant challenge for blockchain forensics, as funds entering a mixer deliberately break the transaction trail. A future version of Ethos could detect when funds enter a mixer contract and flag it as a critical event in the graph, even if the trail beyond the mixer cannot be traced. This would then allow investigators to identify when a trail goes cold, which is highly valuable in a forensic investigation.

### ***Integrated graph visualisation:***

The current workflow requires the user to open the exported GEXF file in a separate program, such as Gephi. Integrating an in-tool visualisation layer, such as a browser-rendered interactive graph through other Python libraries, would remove this dependency and also further streamline the investigation

workflow. It would make the tool accessible to investigators without prior experience with external software and deliver the visual output as part of the tool's functionality rather than a post-use step.

## References

Chainalysis (2025) The Chainalysis 2025 Crypto Crime Report. Available at:

<https://www.chainalysis.com/blog/2025-crypto-crime-report/> (Accessed: 21 April 2026)

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) Introduction to Algorithms. 4th edn. MIT Press.

Ethereum.org (no date) Transactions. Available at: <https://ethereum.org/en/developers/docs/transactions/> (Accessed: 21 April 2026)

Etherscan (no date-a) Get a list of 'Normal' Transactions by Address. Available at:

<https://docs.etherscan.io/api-endpoints/accounts> (Accessed: 21 April 2026)

Etherscan (no date-b) Get Contract Source Code for Verified Contract Source Codes. Available at:

<https://docs.etherscan.io/api-endpoints/contracts> (Accessed: 21 April 2026)

Etherscan (no date-c) Rate Limits. Available at: <https://docs.etherscan.io/resources/rate-limits> (Accessed: 21 April 2026)

FATF (2020) Virtual Assets Red Flag Indicators of Money Laundering and Terrorist Financing. Paris: FATF. Available at: <https://www.fatf-gafi.org/content/dam/fatf-gafi/reports/Virtual-Assets-Red-Flag-Indicators.pdf> (Accessed: 21 April 2026)

NetworkX Developers (2024) NetworkX documentation. Available at:

<https://networkx.org/documentation/stable/> (Accessed: 21 April 2026)

Python Software Foundation (no date) argparse – Parser for command-line options, arguments and sub-commands. Available at: <https://docs.python.org/3/library/argparse.html> (Accessed: 21 April 2026)

Reitz, K. (no date) Requests: HTTP for Humans™. Available at: <https://requests.readthedocs.io/> (Accessed: 21 April 2026)

# Appendices

## Appendix A - Full Terminal Output from Primary Investigation Run

PS B:\ethos> python ethos.py -a 0xDdA24103e4REDACTED

```
_____ --
|_____|
||_|_|_|_|_|
|_|_|_|_|_|_|_|_|_|
|_|_|_|_|_|_|_|_|_|_|
|_|_|_|_|_|_|_|_|_|_|
|_|_|_|_|_|_|_|_|_|_|
|_|_|_|_|_|_|_|_|_|_|
```

### Ethos V1 - Illuminating the Ledger

^

/^

/^

|-----|

\/

\/

v

[!] INVESTIGATION START: 2026-05-02 07:57:32

[!] Target Address: 0xdda24103e4REDACTED

[!] Config: 3 Hops | 20 Tx Limit per node

[\*] Initial Scan: Found 20 transactions leaving target.

[Link] Source -> 0xfaf2f8ded9... | Amount: 0.0128 ETH

[Link] Source -> 0xad8fca418e... | Amount: 5.2000 ETH

[Link] Source -> 0x8dc0b7b50d... | Amount: 5.5400 ETH

[Link] Source -> 0x43a7e39279... | Amount: 5.0000 ETH

[Link] Source -> 0xf06587b7ff... | Amount: 5.8639 ETH

[Link] Source -> 0x06fecb1599... | Amount: 5.0000 ETH

[Link] Source -> 0x2ce01beeed... | Amount: 5.0000 ETH

[Link] Source -> 0xa4724e2b09... | Amount: 5.0000 ETH

[Link] Source -> 0x48551b6e1f... | Amount: 0.0823 ETH

[Link] Source -> 0xdfddeddb5f... | Amount: 5.0000 ETH

[Link] Source -> 0xc4176426f2... | Amount: 5.0000 ETH

[Link] Source -> 0x85316fe3e9... | Amount: 5.0000 ETH

[Link] Source -> 0x8fab61b56... | Amount: 5.0000 ETH

[Link] Source -> 0x5540b26f44... | Amount: 5.0000 ETH

[Link] Source -> 0xce912cdb25... | Amount: 5.0000 ETH

[Link] Source -> 0x2b9988f16b... | Amount: 5.0000 ETH

[Link] Source -> 0x3fc91a3afd... | Amount: 0.0000 ETH

[Link] Source -> 0x45804880de... | Amount: 0.0000 ETH

[Link] Source -> 0x3fc91a3afd... | Amount: 0.0000 ETH

[Link] Source -> 0x2402253996... | Amount: 0.0000 ETH

[\*] Processing Hop 1: 0xfaf2f8ded954...

[Link] 0xfaf2f8... -> 0x077d36... | 0.0116 ETH

[\*] Processing Hop 1: 0xad8fca418eb8...

[Link] 0xad8fca... -> 0x18c4bf... | 5.1990 ETH

[\*] Processing Hop 1: 0x8dc0b7b50db2...

[Link] 0x8dc0b7... -> 0x754ab5... | 5.5395 ETH

[\*] Processing Hop 1: 0x43a7e39279d1...

[Link] 0x43a7e3... -> 0x18c4bf... | 4.9993 ETH

[\*] Processing Hop 1: 0xf06587b7ff1e...

[Link] 0xf06587... -> 0xa9d1e0... | 1.0000 ETH

[Link] 0xf06587... -> 0xb5d85c... | 0.3685 ETH

[Link] 0xf06587... -> 0xa9d1e0... | 0.4856 ETH

[Link] 0xf06587... -> 0xb5d85c... | 0.5000 ETH

[Link] 0xf06587... -> 0xa9d1e0... | 0.5000 ETH

[Link] 0xf06587... -> 0xb5d85c... | 0.2000 ETH

[Link] 0xf06587... -> 0xa9d1e0... | 0.5000 ETH

[\*] Processing Hop 1: 0x06fecb159974...

[Link] 0x06fecb... -> 0xee7f2d... | 4.9995 ETH

[\*] Processing Hop 1: 0x2ce01beeed98...

[Link] 0x2ce01b... -> 0xee7f2d... | 4.9996 ETH

[\*] Processing Hop 1: 0xa4724e2b09c2...

[Link] 0xa4724e... -> 0x18c4bf... | 4.9995 ETH

[\*] Processing Hop 1: 0x48551b6e1f85...

[Link] 0x48551b... -> 0xee462e... | 0.0035 ETH

[Link] 0x48551b... -> 0x6b1754... | 0.0000 ETH

[Link] 0x48551b... -> 0x3fc91a... | 0.5508 ETH

[Link] 0x48551b... -> 0xcc12f... | 23.3365 ETH

[Link] 0x48551b... -> 0x609c69... | 0.0041 ETH

[Link] 0x48551b... -> 0x609c69... | 0.0041 ETH

[Link] 0x48551b... -> 0xe41d24... | 0.0000 ETH

[Link] 0x48551b... -> 0x3fc91a... | 0.3900 ETH

[Link] 0x48551b... -> 0x3fc91a... | 0.0000 ETH

[Link] 0x48551b... -> 0x6a851c... | 0.0048 ETH

[Link] 0x48551b... -> 0x6a851c... | 0.0024 ETH

[Link] 0x48551b... -> 0xd1c55f... | 0.0145 ETH

[Link] 0x48551b... -> 0xdac17f... | 0.0000 ETH

[Link] 0x48551b... -> 0xd5bb51... | 7.4660 ETH

[Link] 0x48551b... -> 0xf3d6dd... | 0.0727 ETH

[Link] 0x48551b... -> 0x3fc91a... | 5.0000 ETH

[Link] 0x48551b... -> 0xc7fb14... | 0.0718 ETH

[\*] Processing Hop 1: 0xdfddeddb5f30...

[Link] 0xdfdded... -> 0xee7f2d... | 4.9995 ETH

[\*] Processing Hop 1: 0xc4176426f2fa...

[Link] 0xc41764... -> 0x18c4bf... | 4.9995 ETH

[\*] Processing Hop 1: 0x85316fe3e979...

[Link] 0x85316f... -> 0x18c4bf... | 4.9992 ETH

[\*] Processing Hop 1: 0x8fab61b56aa...

[Link] 0x8fab6... -> 0x18c4bf... | 4.9994 ETH

[\*] Processing Hop 1: 0x5540b26f4432...

[Link] 0x5540b2... -> 0x18c4bf... | 4.9991 ETH

[\*] Processing Hop 1: 0xce912cdb2502...

[Link] 0xce912c... -> 0x18c4bf... | 4.9992 ETH

[\*] Processing Hop 1: 0x2b9988f16b37...

[Link] 0x2b9988... -> 0xee7f2d... | 4.9990 ETH

[\*] Processing Hop 1: 0x3fc91a3afd70...

[\*] Processing Hop 1: 0x45804880de22...

[\*] Processing Hop 1: 0x240225399633...

[\*] Processing Hop 2: 0x077d360f11d2...

[\*] Processing Hop 2: 0x18c4bf7c4700...

[Link] 0x18c4bf... -> 0xef3a93... | 0.0000 ETH

[Link] 0x18c4bf... -> 0xbbbbca... | 0.0000 ETH

[Link] 0x18c4bf... -> 0x7fc665... | 0.0000 ETH

[Link] 0x18c4bf... -> 0xa1faa1... | 0.0000 ETH

[Link] 0x18c4bf... -> 0x36e66f... | 0.0000 ETH

[Link] 0x18c4bf... -> 0x3593d1... | 0.0000 ETH

[Link] 0x18c4bf... -> 0x3073f7... | 0.0000 ETH

[Link] 0x18c4bf... -> 0x61e90a... | 0.0000 ETH

[Link] 0x18c4bf... -> 0x4cc193... | 0.0000 ETH

[Link] 0x18c4bf... -> 0x1ceb5c... | 0.0000 ETH

[Link] 0x18c4bf... -> 0x27054b... | 0.0000 ETH

[Link] 0x18c4bf... -> 0x4fabb1... | 0.0000 ETH

[Link] 0x18c4bf... -> 0xa9b1eb... | 0.0000 ETH

[Link] 0x18c4bf... -> 0x28c6c0... | 0.5643 ETH

[Link] 0x18c4bf... -> 0xaea46a... | 0.0000 ETH

[Link] 0x18c4bf... -> 0xa0b869... | 0.0000 ETH

[Link] 0x18c4bf... -> 0x6b1754... | 0.0000 ETH

[\*] Processing Hop 2: 0x754ab517159a...

[Link] 0x754ab5... -> 0x28c6c0... | 1.4394 ETH

[Link] 0x754ab5... -> 0x28c6c0... | 1.4995 ETH

[Link] 0x754ab5... -> 0x28c6c0... | 0.7995 ETH

[Link] 0x754ab5... -> 0x28c6c0... | 0.6894 ETH



[Link] 0x754ab5... -> 0x28c6c0... | 1.3294 ETH  
[Link] 0x754ab5... -> 0x28c6c0... | 0.9774 ETH  
[Link] 0x754ab5... -> 0x28c6c0... | 1.4995 ETH  
[Link] 0x754ab5... -> 0x28c6c0... | 0.6192 ETH  
[Link] 0x754ab5... -> 0x28c6c0... | 1.4992 ETH  
[Link] 0x754ab5... -> 0x28c6c0... | 0.6992 ETH  
[\*] Processing Hop 2: 0xa9d1e08c7793...  
[\*] Processing Hop 2: 0xb5d85cbf7cb3...  
[\*] Processing Hop 2: 0xee7f2d8257aa...  
[Link] 0xee7f2d... -> 0x814e09... | 0.0000 ETH  
[Link] 0xee7f2d... -> 0xb70526... | 0.0000 ETH  
[Link] 0xee7f2d... -> 0xa1faa1... | 0.0000 ETH  
[Link] 0xee7f2d... -> 0x163f8c... | 0.0000 ETH  
[Link] 0xee7f2d... -> 0x9f8f72... | 0.0000 ETH  
[Link] 0xee7f2d... -> 0x61e90a... | 0.0000 ETH  
[Link] 0xee7f2d... -> 0x1ceb5c... | 0.0000 ETH  
[Link] 0xee7f2d... -> 0x27054b... | 0.0000 ETH  
[Link] 0xee7f2d... -> 0x4cc193... | 0.0000 ETH  
[Link] 0xee7f2d... -> 0x92d6c1... | 0.0000 ETH  
[Link] 0xee7f2d... -> 0x8457ca... | 0.0000 ETH  
[Link] 0xee7f2d... -> 0xf8c352... | 0.0000 ETH  
[Link] 0xee7f2d... -> 0x7dd9c5... | 0.0000 ETH  
[Link] 0xee7f2d... -> 0x6de037... | 0.0000 ETH  
[Link] 0xee7f2d... -> 0x5faa98... | 0.0000 ETH  
[Link] 0xee7f2d... -> 0x2260fa... | 0.0000 ETH  
[Link] 0xee7f2d... -> 0xa1d0e2... | 0.0000 ETH  
[Link] 0xee7f2d... -> 0xf43308... | 0.0000 ETH  
[Link] 0xee7f2d... -> 0x28c6c0... | 0.3144 ETH  
[\*] Processing Hop 2: 0xee462e73da66...  
[\*] Processing Hop 2: 0x6b175474e890...  
[\*] Processing Hop 2: 0xcc12f612c6b...

[Link] 0xccc12f... -> 0x8583ab... | 12.0956 ETH  
[Link] 0xccc12f... -> 0x95bef7... | 3.3800 ETH  
[Link] 0xccc12f... -> 0x6a4c44... | 1.0591 ETH  
[Link] 0xccc12f... -> 0x079e98... | 6.8000 ETH  
[\*] Processing Hop 2: 0x609c690e8f7d...  
[\*] Processing Hop 2: 0xe41d2489571d...  
[\*] Processing Hop 2: 0x6a851ce0ad9d...  
[Link] 0x6a851c... -> 0x0e7d46... | 0.0043 ETH  
[Link] 0x6a851c... -> 0xdac17f... | 0.0000 ETH  
[Link] 0x6a851c... -> 0xdac17f... | 0.0000 ETH  
[\*] Processing Hop 2: 0xd1c55f9d9d16...  
[Link] 0xd1c55f... -> 0x4c77ef... | 0.1476 ETH  
[Link] 0xd1c55f... -> 0x4c77ef... | 0.0093 ETH  
[Link] 0xd1c55f... -> 0x4c77ef... | 0.0176 ETH  
[Link] 0xd1c55f... -> 0xdac17f... | 0.0000 ETH  
[Link] 0xd1c55f... -> 0x4c77ef... | 0.0141 ETH  
[\*] Processing Hop 2: 0xdac17f958d2e...  
[\*] Processing Hop 2: 0xd5bb51ca30b9...  
[Link] 0xd5bb51... -> 0xe62d14... | 0.0863 ETH  
[\*] Processing Hop 2: 0xf3d6ddcc108f...  
[Link] 0xf3d6dd... -> 0x974caa... | 0.0161 ETH  
[Link] 0xf3d6dd... -> 0x974caa... | 0.0150 ETH  
[Link] 0xf3d6dd... -> 0x974caa... | 0.0176 ETH  
[Link] 0xf3d6dd... -> 0x974caa... | 0.0159 ETH  
[Link] 0xf3d6dd... -> 0x974caa... | 0.0003 ETH  
[Link] 0xf3d6dd... -> 0x2b591e... | 0.0000 ETH  
[Link] 0xf3d6dd... -> 0x974caa... | 0.0195 ETH  
[Link] 0xf3d6dd... -> 0x974caa... | 0.0125 ETH  
[Link] 0xf3d6dd... -> 0x974caa... | 0.0267 ETH  
[Link] 0xf3d6dd... -> 0x974caa... | 0.0029 ETH  
[Link] 0xf3d6dd... -> 0x974caa... | 0.0025 ETH

[\*] Processing Hop 2: 0xc7fb149e1535...

[Link] 0xc7fb14... -> 0x75f47b... | 0.0043 ETH

[Link] 0xc7fb14... -> 0x3dd14b... | 0.0025 ETH

[Link] 0xc7fb14... -> 0x101e26... | 0.0065 ETH

[Link] 0xc7fb14... -> 0xa5836c... | 0.0027 ETH

[Link] 0xc7fb14... -> 0x0ba3d3... | 0.0139 ETH

[Link] 0xc7fb14... -> 0x79400a... | 0.0020 ETH

[Link] 0xc7fb14... -> 0x784de1... | 0.0063 ETH

[Link] 0xc7fb14... -> 0xe04486... | 0.0013 ETH

[Link] 0xc7fb14... -> 0x784de1... | 0.0110 ETH

[Link] 0xc7fb14... -> 0x784de1... | 0.0200 ETH

[Link] 0xc7fb14... -> 0xdf80b1... | 0.0015 ETH

[Link] 0xc7fb14... -> 0xf21676... | 1.0310 ETH

[Link] 0xc7fb14... -> 0x784de1... | 0.0400 ETH

[Link] 0xc7fb14... -> 0x3b30d9... | 0.0074 ETH

[Link] 0xc7fb14... -> 0x0eafe9... | 0.0043 ETH

[Link] 0xc7fb14... -> 0x4bbcce... | 0.0082 ETH

[\*] Processing Hop 3: 0xef3a930e1fff...

[\*] Processing Hop 3: 0xbbbbca6a901c...

[\*] Processing Hop 3: 0x7fc66500c84a...

[\*] Processing Hop 3: 0xa1faa113cbe5...

[\*] Processing Hop 3: 0x36e66fbbce51...

[\*] Processing Hop 3: 0x3593d125a4f7...

[\*] Processing Hop 3: 0x3073f7aaa4db...

[\*] Processing Hop 3: 0x61e90a50137e...

[\*] Processing Hop 3: 0x4cc19356f2d3...

[\*] Processing Hop 3: 0x1ceb5cb57c4d...

[\*] Processing Hop 3: 0x27054b13b1b7...

[\*] Processing Hop 3: 0x4fabbb145d646...

[\*] Processing Hop 3: 0xa9b1eb5908cf...

[\*] Processing Hop 3: 0x28c6c06298d5...

[Link] 0x28c6c0... -> 0xee7ae8... | 0.0000 ETH  
[Link] 0x28c6c0... -> 0xee7ae8... | 0.0000 ETH  
[Link] 0x28c6c0... -> 0x1212cf... | 0.0427 ETH  
[Link] 0x28c6c0... -> 0x771467... | 0.0000 ETH  
[Link] 0x28c6c0... -> 0xee7ae8... | 0.0000 ETH  
[Link] 0x28c6c0... -> 0xee7ae8... | 0.0000 ETH  
[Link] 0x28c6c0... -> 0xbc26bc... | 1.0063 ETH  
[Link] 0x28c6c0... -> 0xee7ae8... | 0.0000 ETH  
[Link] 0x28c6c0... -> 0xee7ae8... | 0.0000 ETH  
[Link] 0x28c6c0... -> 0x338255... | 0.4999 ETH  
[Link] 0x28c6c0... -> 0xee7ae8... | 0.0000 ETH  
[Link] 0x28c6c0... -> 0xee7ae8... | 0.0000 ETH  
[Link] 0x28c6c0... -> 0xee7ae8... | 0.0000 ETH  
[\*] Processing Hop 3: 0xaea46a60368a...  
[\*] Processing Hop 3: 0xa0b86991c621...  
[\*] Processing Hop 3: 0x814e0908b12a...  
[\*] Processing Hop 3: 0xb705268213d5...  
[\*] Processing Hop 3: 0x163f8c246792...  
[\*] Processing Hop 3: 0x9f8f72aa9304...  
[\*] Processing Hop 3: 0x92d6c1e31e14...  
[\*] Processing Hop 3: 0x8457ca5040ad...  
[\*] Processing Hop 3: 0xf8c3527cc043...  
[\*] Processing Hop 3: 0x7dd9c5cba05e...  
[\*] Processing Hop 3: 0x6de037ef9ad2...  
[\*] Processing Hop 3: 0x5faa989af96a...  
[\*] Processing Hop 3: 0x2260fac5e554...  
[\*] Processing Hop 3: 0xa1d0e215a23d...  
[\*] Processing Hop 3: 0xf43308936689...  
[\*] Processing Hop 3: 0x8583abdb6517...  
[Link] 0x8583ab... -> 0xf1da17... | 12.0952 ETH  
[\*] Processing Hop 3: 0x95bef7d80918...

[Link] 0x95bef7... -> 0xf1da17... | 3.3796 ETH

[\*] Processing Hop 3: 0x6a4c4432ca09...

[Link] 0x6a4c44... -> 0x5d73d3... | 0.0168 ETH

[Link] 0x6a4c44... -> 0x3ad69a... | 0.0883 ETH

[Link] 0x6a4c44... -> 0x80bba8... | 0.0029 ETH

[Link] 0x6a4c44... -> 0xdbbd72... | 0.1512 ETH

[Link] 0x6a4c44... -> 0xdbbd72... | 9.3968 ETH

[Link] 0x6a4c44... -> 0x65489d... | 0.4455 ETH

[Link] 0x6a4c44... -> 0xcdf8ac... | 0.5303 ETH

[\*] Processing Hop 3: 0x079e98fbe408...

[Link] 0x079e98... -> 0xf1da17... | 6.7995 ETH

[\*] Processing Hop 3: 0x0e7d4697ffd7...

[Link] 0x0e7d46... -> 0x8f28ee... | 0.1092 ETH

[Link] 0x0e7d46... -> 0x264d5e... | 0.3069 ETH

[Link] 0x0e7d46... -> 0x264d5e... | 0.3175 ETH

[Link] 0x0e7d46... -> 0x8bceaa... | 0.0986 ETH

[Link] 0x0e7d46... -> 0x4538a9... | 0.0494 ETH

[\*] Processing Hop 3: 0x4c77efcb44ee...

[Link] 0x4c77ef... -> 0xe13cbd... | 1.3778 ETH

[Link] 0x4c77ef... -> 0x698250... | 0.0000 ETH

[Link] 0x4c77ef... -> 0x698250... | 0.0000 ETH

[Link] 0x4c77ef... -> 0x698250... | 0.0000 ETH

[Link] 0x4c77ef... -> 0x698250... | 0.0000 ETH

[Link] 0x4c77ef... -> 0x698250... | 0.0000 ETH

[Link] 0x4c77ef... -> 0x698250... | 0.0000 ETH

[Link] 0x4c77ef... -> 0x698250... | 0.0000 ETH

[Link] 0x4c77ef... -> 0x698250... | 0.0000 ETH

[Link] 0x4c77ef... -> 0x698250... | 0.0000 ETH

[\*] Processing Hop 3: 0xe62d14ade57d...

[\*] Processing Hop 3: 0x974caa59e496...

[Link] 0x974caa... -> 0x20453f... | 0.0006 ETH

[Link] 0x974caa... -> 0xa52e0f... | 0.0006 ETH

[Link] 0x974caa... -> 0x27037c... | 0.0006 ETH  
[Link] 0x974caa... -> 0x29d5b4... | 0.0006 ETH  
[Link] 0x974caa... -> 0xbe27bc... | 0.0005 ETH  
[Link] 0x974caa... -> 0xfa913e... | 0.0005 ETH  
[Link] 0x974caa... -> 0x1056b3... | 0.0006 ETH  
[Link] 0x974caa... -> 0x61c9d8... | 0.0194 ETH  
[Link] 0x974caa... -> 0x1c3e3d... | 0.0006 ETH  
[Link] 0x974caa... -> 0x8fac54... | 0.0510 ETH  
[\*] Processing Hop 3: 0x2b591e99afe9...  
[\*] Processing Hop 3: 0x75f47b53148f...  
[\*] Processing Hop 3: 0x3dd14b090846...  
[Link] 0x3dd14b... -> 0xf52605... | 0.0258 ETH  
[Link] 0x3dd14b... -> 0xf52605... | 0.0050 ETH  
[Link] 0x3dd14b... -> 0xf52605... | 0.0025 ETH  
[Link] 0x3dd14b... -> 0xf52605... | 0.0050 ETH  
[Link] 0x3dd14b... -> 0xf52605... | 0.0200 ETH  
[Link] 0x3dd14b... -> 0xf52605... | 0.0102 ETH  
[Link] 0x3dd14b... -> 0xf52605... | 0.0016 ETH  
[Link] 0x3dd14b... -> 0xf52605... | 0.0108 ETH  
[Link] 0x3dd14b... -> 0xf52605... | 0.0340 ETH  
[Link] 0x3dd14b... -> 0xf52605... | 0.0521 ETH  
[\*] Processing Hop 3: 0x101e26602770...  
[Link] 0x101e26... -> 0xb21b7c... | 0.2000 ETH  
[Link] 0x101e26... -> 0xb21b7c... | 0.0464 ETH  
[Link] 0x101e26... -> 0xb21b7c... | 0.1905 ETH  
[Link] 0x101e26... -> 0xb21b7c... | 0.1926 ETH  
[Link] 0x101e26... -> 0xb21b7c... | 0.0039 ETH  
[Link] 0x101e26... -> 0xb21b7c... | 0.0283 ETH  
[\*] Processing Hop 3: 0xa5836ce898de...  
[Link] 0xa5836c... -> 0x8a9e0f... | 0.0027 ETH  
[\*] Processing Hop 3: 0x0ba3d381da96...

[Link] 0x0ba3d3... -> 0x96d047... | 0.0012 ETH  
[Link] 0x0ba3d3... -> 0x96d047... | 0.0102 ETH  
[Link] 0x0ba3d3... -> 0x96d047... | 0.0025 ETH  
[Link] 0x0ba3d3... -> 0x96d047... | 0.0039 ETH  
[Link] 0x0ba3d3... -> 0x96d047... | 0.0101 ETH  
[Link] 0x0ba3d3... -> 0x96d047... | 0.0061 ETH  
[Link] 0x0ba3d3... -> 0x96d047... | 0.0067 ETH  
[Link] 0x0ba3d3... -> 0x96d047... | 0.0029 ETH  
[Link] 0x0ba3d3... -> 0x96d047... | 0.0032 ETH  
[Link] 0x0ba3d3... -> 0x96d047... | 0.0059 ETH  
[\*] Processing Hop 3: 0x79400a891f52...  
[Link] 0x79400a... -> 0xeba881... | 0.0020 ETH  
[\*] Processing Hop 3: 0x784de151dbe1...  
[Link] 0x784de1... -> 0xcffad3... | 0.2737 ETH  
[Link] 0x784de1... -> 0xcffad3... | 0.1224 ETH  
[Link] 0x784de1... -> 0xcffad3... | 0.0063 ETH  
[Link] 0x784de1... -> 0xcffad3... | 0.0110 ETH  
[Link] 0x784de1... -> 0xcffad3... | 0.0200 ETH  
[\*] Processing Hop 3: 0xe04486912384...  
[Link] 0xe04486... -> 0x666e24... | 0.0072 ETH  
[Link] 0xe04486... -> 0x666e24... | 0.0012 ETH  
[Link] 0xe04486... -> 0x666e24... | 0.0084 ETH  
[Link] 0xe04486... -> 0x39ea8a... | 0.0007 ETH  
[Link] 0xe04486... -> 0x4419b5... | 0.0010 ETH  
[Link] 0xe04486... -> 0x666e24... | 0.0039 ETH  
[Link] 0xe04486... -> 0x666e24... | 0.0006 ETH  
[Link] 0xe04486... -> 0x666e24... | 0.0017 ETH  
[\*] Processing Hop 3: 0xdf80b1595fd8...  
[Link] 0xdf80b1... -> 0x29bc60... | 0.0041 ETH  
[Link] 0xdf80b1... -> 0xc84235... | 0.0137 ETH  
[Link] 0xdf80b1... -> 0xc84235... | 0.0316 ETH

[Link] 0xdf80b1... -> 0xc84235... | 0.0142 ETH

[Link] 0xdf80b1... -> 0xc84235... | 0.0332 ETH

[Link] 0xdf80b1... -> 0xc84235... | 0.0104 ETH

[\*] Processing Hop 3: 0xf216762b371d...

[Link] 0xf21676... -> 0x332b42... | 0.0860 ETH

[Link] 0xf21676... -> 0x694605... | 0.0000 ETH

[Link] 0xf21676... -> 0xf1ac55... | 0.0004 ETH

[Link] 0xf21676... -> 0xf1ac55... | 0.0036 ETH

[Link] 0xf21676... -> 0x67a76d... | 0.0100 ETH

[Link] 0xf21676... -> 0x0a09c4... | 0.0657 ETH

[Link] 0xf21676... -> 0x694605... | 0.0116 ETH

[Link] 0xf21676... -> 0xf1ac55... | 0.0030 ETH

[Link] 0xf21676... -> 0x0a09c4... | 0.0520 ETH

[Link] 0xf21676... -> 0xf1ac55... | 0.0077 ETH

[\*] Processing Hop 3: 0x3b30d90d7af3...

[Link] 0x3b30d9... -> 0xb8001c... | 0.0073 ETH

[Link] 0x3b30d9... -> 0xb8001c... | 0.0045 ETH

[\*] Processing Hop 3: 0x0eafe9084169...

[Link] 0x0eafe9... -> 0x9d9827... | 0.0360 ETH

[Link] 0x0eafe9... -> 0x9d9827... | 0.2700 ETH

[Link] 0x0eafe9... -> 0x9d9827... | 0.1934 ETH

[Link] 0x0eafe9... -> 0x7d05fa... | 0.0030 ETH

[Link] 0x0eafe9... -> 0x0ea1a2... | 0.0010 ETH

[\*] Processing Hop 3: 0x4bbcce510d1e...

[Link] 0x4bbcce... -> 0x72bee3... | 0.0016 ETH

[Link] 0x4bbcce... -> 0xce3a52... | 0.0066 ETH

=====

### --- FINAL INVESTIGATION SUMMARY ---

Total Unique Addresses Found: 88

Total Transactions Mapped: 1308

Total ETH Flow from Target: 71.6989 ETH



-----

Files Created in 'forensic\_results/':

-> 0xdda24103\_20260502\_0803\_nodes.csv

-> 0xdda24103\_20260502\_0803\_edges.csv

-> 0xdda24103\_20260502\_0803\_graph.gexf

=====

PS B:\ethos>